

Docker 运维速查表

面向面试与生产排障，按使用频率排序

一、容器生命周期

操作	命令	说明
后台运行	<code>docker run -d --name web -p 80:80 nginx:alpine</code>	-d 后台, --name 命名
交互运行	<code>docker run -it --rm ubuntu:22.04 bash</code>	-it 交互终端, --rm 退出即删
查看运行中	<code>docker ps</code>	只显示存活容器
查看全部	<code>docker ps -a</code>	含已退出
只看 ID	<code>docker ps -q</code>	用于批量操作
按状态过滤	<code>docker ps -a -f status=exited</code>	过滤已退出
停止	<code>docker stop web</code>	发 SIGTERM, 优雅停止
强制停止	<code>docker kill web</code>	发 SIGKILL
启动	<code>docker start web</code>	启动已停止容器
重启	<code>docker restart web</code>	重启容器
删除	<code>docker rm web</code>	删已停止的
强制删除	<code>docker rm -f web</code>	运行中的也强制删
批量删已退出	<code>docker rm \$(docker ps -aq -f status=exited)</code>	清理僵尸容器
查看详情	<code>docker inspect web</code>	JSON 格式详细信息
查看进程	<code>docker top web</code>	容器内进程列表
查看资源	<code>docker stats web --no-stream</code>	CPU/内存/IO/网络

二、进入容器排障

场景	命令	判断方法
Debian/Ubuntu/CentOS	<code>docker exec -it web /bin/bash</code>	有 bash
Alpine/BusyBox	<code>docker exec -it web /bin/sh</code>	无 bash, 只有 sh
不确定时	<code>docker exec web ls /bin/ \& grep -E "bash\ sh"</code>	先探测
不进入执行命令	<code>docker exec web cat /etc/nginx/nginx.conf</code>	直接执行
用 nsenter 绕过	<code>nsenter -t PID -n -m -u -i -p sh</code>	docker exec 失效时用

面试点：Alpine 基于 musl libc, 镜像小但无 bash, 生产排障用 sh

三、日志管理

需求	命令
全部日志	<code>docker logs web</code>
实时跟踪	<code>docker logs -f web</code>
最后 N 行	<code>docker logs --tail 50 web</code>
最近时间	<code>docker logs --since 10m web</code>
时间戳	<code>docker logs -t web</code>
标准错误	<code>docker logs web 2>&1 \ grep ERROR</code>

四、镜像操作

操作	命令
搜索	<code>docker search nginx</code>
拉取	<code>docker pull nginx:alpine</code>
查看本地	<code>docker images</code>
查看详情	<code>docker inspect nginx:alpine</code>
删除镜像	<code>docker rmi nginx:alpine</code>
强制删除	<code>docker rmi -f nginx:alpine</code>
清理悬空镜像	<code>docker image prune</code>
大清理	<code>docker system prune -a</code>
构建	<code>docker build -t myapp:v1 .</code>
指定 Dockerfile	<code>docker build -t myapp:v1 -f Dockerfile.prod .</code>
打标签	<code>docker tag myapp:v1 myapp:latest</code>
推送仓库	<code>docker push registry/myapp:v1</code>
保存为 tar	<code>docker save -o myapp.tar myapp:v1</code>
从 tar 加载	<code>docker load -i myapp.tar</code>

五、数据卷与文件

操作	命令
创建卷	<code>docker volume create my-vol</code>
查看卷	<code>docker volume ls</code>
查看详情	<code>docker volume inspect my-vol</code>
使用 named volume	<code>docker run -d -v my-vol:/data nginx:alpine</code>
使用 bind mount	<code>docker run -d -v /host/path:/container/path nginx:alpine</code>
只读挂载	<code>docker run -d -v /host:/container:ro nginx:alpine</code>
容器间共享	<code>docker run -d --volumes-from web nginx:alpine</code>
宿主机 → 容器	<code>docker cp ./app.conf web:/etc/nginx/conf.d/</code>

Table 5 – continued

操作	命令
容器 → 宿主机	<code>docker cp web:/etc/nginx/nginx.conf ./</code>
删除卷	<code>docker volume rm my-vol</code>
清理未使用卷	<code>docker volume prune</code>

面试题：bind mount 直接映射宿主机路径；named volume 由 Docker 管理，可跨容器共享、可备份

六、网络管理

操作	命令
查看网络	<code>docker network ls</code>
查看详情	<code>docker network inspect bridge</code>
创建网络	<code>docker network create my-net</code>
指定网络运行	<code>docker run -d --network my-net --name db mysql</code>
容器间通信	<code>docker run -d --network my-net --name web nginx</code> → web 内 ping db 通
给运行中容器加网	<code>docker network connect my-net web</code>
断开网络	<code>docker network disconnect my-net web</code>
删除网络	<code>docker network rm my-net</code>
查看容器 IP	<code>docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' web</code>

网络模式	说明
bridge	默认，NAT 方式，容器间隔离
host	共享宿主机网络栈，性能最好
none	无网络，完全隔离
overlay	跨主机通信，Swarm/K8s 用

七、资源限制 (cgroup)

资源	命令	面试关联
CPU 限制	<code>docker run -d --cpus="1.5" nginx:alpine</code>	最多用 1.5 核
CPU 绑定	<code>docker run -d --cpuset-cpus="0,2" nginx:alpine</code>	绑特定物理核

Table 8 – continued

资源	命令	面试关联
CPU 权重	<code>docker run -d --cpu-shares=512 nginx:alpine</code>	相对权重，默认 1024
内存限制	<code>docker run -d --memory="512m" nginx:alpine</code>	硬限制
内存 + 交换	<code>docker run -d --memory="512m" --memory-swap="1g" nginx:alpine</code>	swap 是 memory 的倍数
OOM 行为	<code>docker run -d --oom-kill-disable nginx:alpine</code>	慎用，可能拖垮宿主机
实时看资源	<code>docker stats</code>	所有容器
单次查看	<code>docker stats web --no-stream</code>	用于脚本采集

面试点：--memory 限制的是 **RSS + Cache**，不是单纯物理内存；OOM 时看 `dmesg \ | grep "killed process"`

八、Docker Compose

操作	命令
后台启动	<code>docker compose up -d</code>
强制构建启动	<code>docker compose up -d --build</code>
查看状态	<code>docker compose ps</code>
实时日志	<code>docker compose logs -f</code>
指定服务日志	<code>docker compose logs -f web</code>
进入服务容器	<code>docker compose exec web sh</code>
重启服务	<code>docker compose restart web</code>
构建指定服务	<code>docker compose build web</code>
停止并删除	<code>docker compose down</code>
连卷一起删	<code>docker compose down -v</code>
扩容	<code>docker compose up -d --scale web=3</code>
拉取最新镜像	<code>docker compose pull</code>

九、生产排障三板斧

容器起不来

`docker logs container-id` [# 看报错](#)

```
docker inspect container-id          # 看 ExitCode、Mounts、Env
docker events                        # 看守护进程事件流
docker inspect -f '{{.State.Pid}}' web # 获取 PID 用 nsenter
```

容器 OOM

```
dmesg | grep "killed process"      # 内核 OOM 日志
docker inspect web | grep -i oom   # 看 OOMKilled 状态
docker stats web --no-stream       # 确认内存使用
```

网络不通

```
docker exec web ip addr            # 看容器 IP
docker exec web ping target        # 容器内测试
docker network inspect my-net      # 看网络配置
docker exec web ss -tlnp           # 看监听端口
iptables -t nat -L -n | grep 8080 # 看宿主机 NAT 规则
```

存储满了

```
df -h /var/lib/docker              # Docker 数据目录空间
docker system df                   # Docker 磁盘使用概况
docker system prune -a             # 清理未使用资源
docker volume prune                # 清理未使用卷
```

十、面试高频参数速记

参数	含义	使用场景
-d	detached, 后台运行	生产服务
-it	交互 + 伪终端	进容器调试
--rm	退出自动删除容器	临时测试
-p	端口映射 宿主机: 容器	暴露服务
-P	暴露 Dockerfile 中 EXPOSE 的所有端口	快速测试
-v	挂载卷	数据持久化
--network	指定网络	容器间通信
--restart always	自动重启策略	生产高可用
--memory	内存限制	防资源耗尽
--cpus	CPU 限制	资源隔离
--env / -e	环境变量	配置注入
--env-file	从文件读环境变量	批量配置
--entrypoint	覆盖默认入口	调试时覆盖
--privileged	授予全部权限	危险，尽量不用
--cap-add	添加特定能力	精细化权限

十一、常用组合命令

```
# 一键清理所有停止的容器、悬空镜像、未使用网络
docker system prune -a -f

# 批量停止所有容器
docker stop $(docker ps -aq)

# 批量删除所有容器（包括运行的）
docker rm -f $(docker ps -aq)

# 查看容器启动命令
docker inspect -f '{{.Config.Cmd}} {{.Config.Entrypoint}}' web

# 查看容器环境变量
docker inspect -f '{{range .Config.Env}}{{.}}\n{{end}}' web

# 查看容器挂载
docker inspect -f '{{range .Mounts}}{{.Source}} -> {{.Destination}}\n{{end}}' web

# 查看最后创建的 3 个容器
docker ps -a --format "table {{.Names}}\t{{.Status}}\t{{.Image}}" | head -n 4
```

打印或保存到本地，面试前过一遍，生产排障直接查